

**“Cvyt”**  
v0.0.1

# Table of Contents

|                                      |    |
|--------------------------------------|----|
| Chapter 1.....                       | 3  |
| What it is this.....                 | 3  |
| Current version of app.....          | 3  |
| Limitations.....                     | 3  |
| Config file.....                     | 3  |
| Modules.....                         | 3  |
| License.....                         | 3  |
| Goals.....                           | 3  |
| Tags.....                            | 3  |
| Supported platform.....              | 4  |
| Used packages, dependencies.....     | 4  |
| Python should be.....                | 4  |
| Chapter 2.....                       | 5  |
| How to use it.....                   | 5  |
| UI.....                              | 5  |
| Main window – left overview.....     | 5  |
| Main menu.....                       | 6  |
| Main.....                            | 6  |
| Update.....                          | 6  |
| Module content.....                  | 6  |
| How to.....                          | 6  |
| Config.....                          | 8  |
| Config tree.....                     | 10 |
| Supported type.....                  | 11 |
| Add.....                             | 11 |
| Examples.....                        | 11 |
| Edit.....                            | 12 |
| Remove.....                          | 12 |
| Config tree, value combinations..... | 13 |
| Add.....                             | 13 |
| Edit.....                            | 13 |
| Remove.....                          | 14 |
| Help tab.....                        | 14 |
| Info tab.....                        | 14 |
| Chapter 3.....                       | 16 |
| Modules.....                         | 16 |
| Module content structure.....        | 16 |
| Module import/start.....             | 16 |
| Update module.....                   | 16 |
| Chapter 4.....                       | 17 |
| Troubleshooting.....                 | 17 |
| Logging.....                         | 17 |
| Everything si broken.....            | 17 |
| Chapter 5.....                       | 18 |
| Development.....                     | 18 |
| Tests.....                           | 18 |
| Changes, new ideas, cooperation..... | 18 |
| Future.....                          | 18 |
| Technical debt & New ideas.....      | 18 |

# Chapter 1

## What it is this

This is a simple app designed to help you use **opencv-python**. The app provides an easy-to-use UI that allows you to **directly** open and use your own modules, which share the same dependencies (at least for now).

## Current version of the app

- sync
- dynamic import of the module

## Limitations

### *Config file*

The config file is expected to be in the **JSON** format, which comes with a few limitations

- the key must always be a **string** (no numbers or advanced types are allowed)
- value basic options
  - integer
  - float
  - string
- you can also use the following as values
  - dictionary (the key in the dictionary must be a string)
  - list

## *Modules*

You have to specify widget main class in one of three possible ways, otherwise module won't be imported.

- “class\_name” (module\_config.json)
- using “\_\_all\_\_” (\*widget.py file)
- via the “\_\_init\_\_.py” file

## License

GPLv3

## Goals

Playing around with the Qt(pyside6), exploring its pros and cons—though I'm still working on filling in the gaps in my knowledge.

## Tags

#noai #novibecoding #simplepython

## Supported platform

This app is designed to run in a **Linux** environment(for now).

## Used packages, dependencies

*Refer to the **pyproject.toml** file for a full list of dependencies. Currently, the app only requires the basic packages: **python**, **opencv-python**, and **pyside6**.*

## Python

$\geq 3.12$  and  $< 3.15$

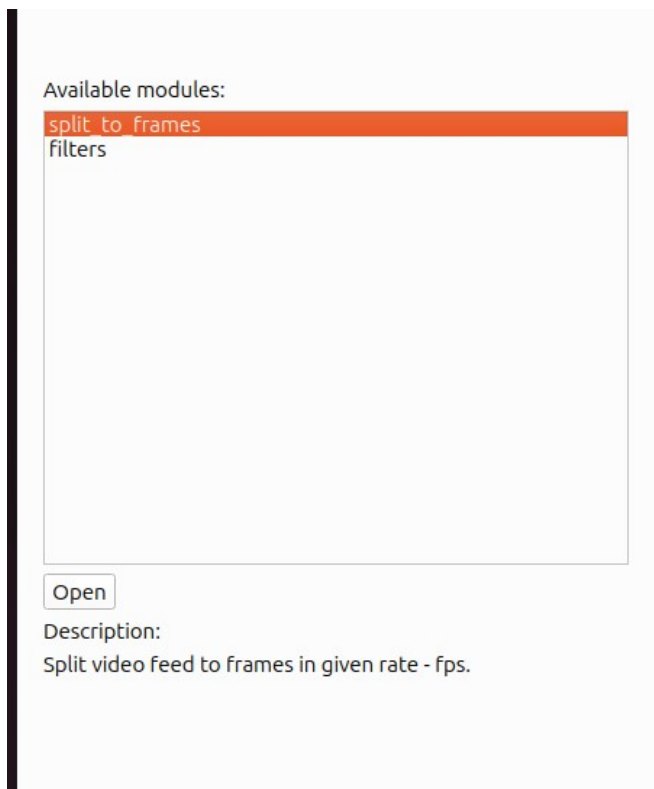
# Chapter 2

## How to use it

It's up to you how you choose to use the app (e.g., as an **.exe** file or running raw Python). The simplest way is to just start it and let it run.

## UI

### *Main window – left panel overview*



The **Overview** contains a list of available modules ("*Available Modules*"). To view the description of a specific module, simply click on its name (once), and the **Description** will appear below the list.

To open a module in a new tab, you have two options

1. Select the desired module (click on its name) and then click the **"Open"** button.
2. Select the module and **double-click** its name.

## Main menu



**Main** = contains the "**Exit**" option to close the app.

**Update** = opens a new tab where you can add more modules to be used

**Config** = opens the config tab, allowing you to modify the app's config file or the config file for your module

**Info** = opens the info tab. This tab will remain open even if all other tabs are closed.

**Help** = opens the help tab or the default browser with the "**HOWTO.pdf**" document for this app. If you prefer to open it in a browser instead of a tab, this can be changed in the app's config file (the default is opening in a tab)

## Main

This option allows you to **Exit** the app.

## Update

You can update, add, or overwrite modules for the **cvyt** app by preparing a simple archive with the correct structure.

```
- modules
|- filters
  |- main_widget.py
  |- logic.py
  |- help.pdf
  |- module_config.json
```

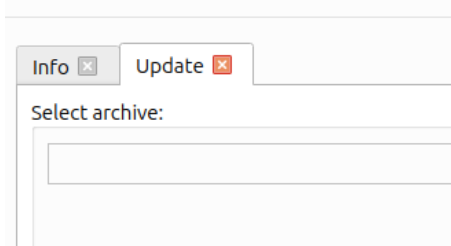
All of this should be included in a simple **modules.zip** archive.

## Module content

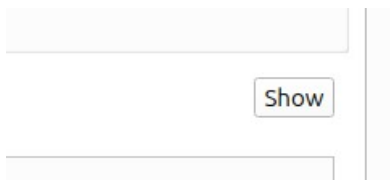
For the most **up-to-date** information, refer to the **cvyt-modules** repository.

## How to

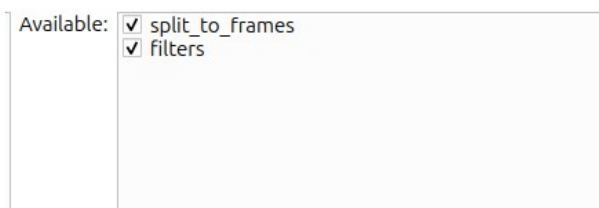
Select the archive by clicking the "**Folder**" button.



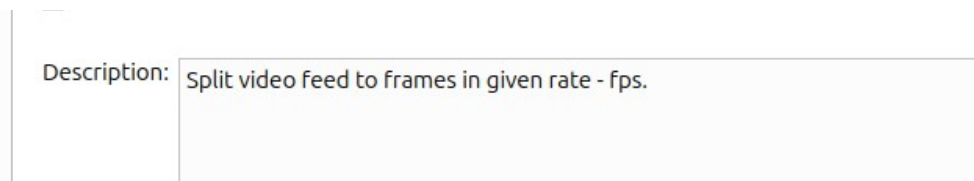
The next step is to validate the contents of the archive (check the folder structure). Simply click the **"Show"** button.

A screenshot of a web interface. It features a light gray background with a vertical line on the right side. On the left, there are two horizontal input fields. Between these fields, centered, is a button with the text "Show" in a dark gray font.

After that, you should see something like this.

A screenshot of a web interface. It shows a section labeled "Available:" followed by a list of two items: "split\_to\_frames" and "filters". Each item has a small checkmark icon to its left, indicating they are selected. The background is light gray with a vertical line on the right.

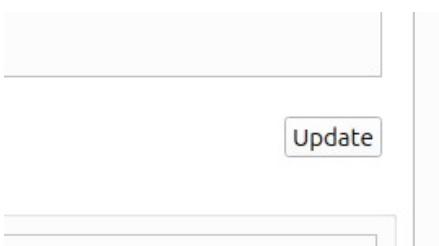
In this example, the archive contains two valid modules: **"split\_to\_frames"** and **"filters"**. To view a simple description of a module (taken from the **"module\_config.json"** file), simply click on the module name.

A screenshot of a web interface. It shows a section labeled "Description:" followed by a text box containing the text "Split video feed to frames in given rate - fps." The background is light gray with a vertical line on the right.

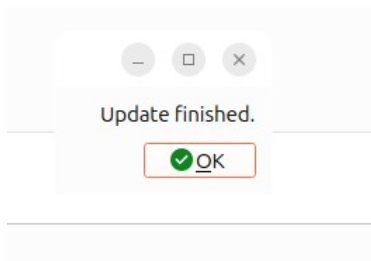
If you have an archive with many modules and don't want to manually check each one, you can use the **"Select All"** option. By default, all modules are pre-selected for updating.

A screenshot of a web interface. It shows a section with a "Select all" option, which is checked with a small checkmark icon. Below this, there is a section labeled "Description:" followed by a text box containing the text "Split video feed to frames in given rate - fps." The background is light gray with a vertical line on the right.

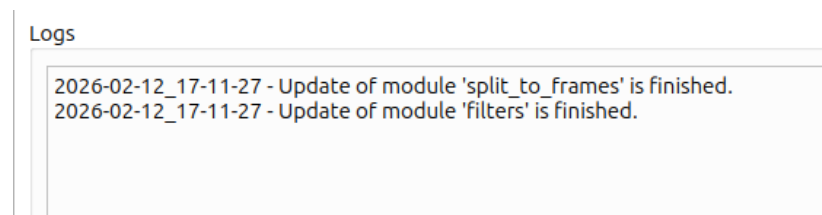
Now it's time to begin the update process.

A screenshot of a web interface. It features a light gray background with a vertical line on the right side. On the left, there are two horizontal input fields. Between these fields, centered, is a button with the text "Update" in a dark gray font.

Once the update is complete, a message will appear indicating whether it was successful or not.



For more detailed information, check the **Logs** box.

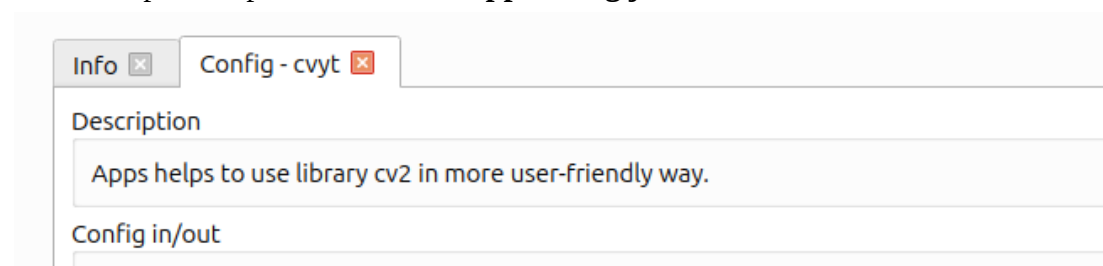


You're almost done! If the update was successful, it's time to restart the app to reload all the modules.

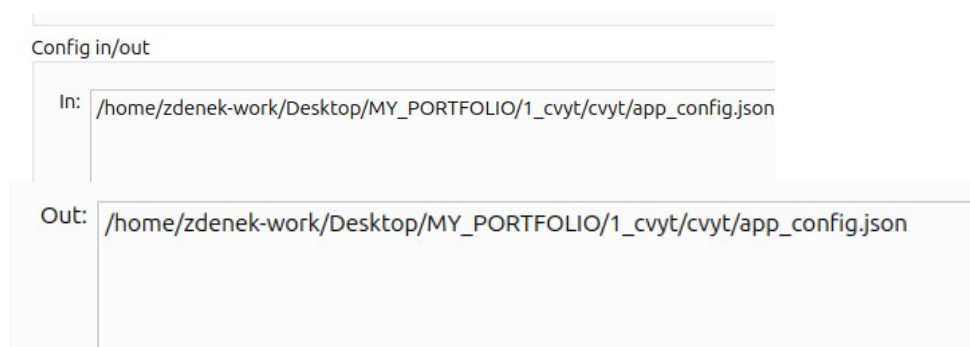
## Config

This tab displays the config file and provides options to modify it. By default, it shows the app's config file, but the same functionality can be used for a custom module (explained later). Simply open the "**Config Tab**" (in the following example, the app's config file).

The description is pulled from the "**app\_config.json**" file.

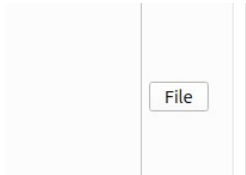


By default, "**config in**" and "**config out**" are set to the paths of the file you are opening in the config tab.





To select a different file, use the **"File"** button (for **"Config in"**). The file will be loaded and displayed immediately after selection. If the file is not a valid JSON file, no **"config tree"** will be shown.



If a valid config file is selected, the **"config tree"** will be visible, displaying the structured information from the config file.

Config

| Name        | Value                                                      |
|-------------|------------------------------------------------------------|
| .           | .                                                          |
| -name       | cvyt                                                       |
| -version    | 0.0.1                                                      |
| -title      | CVYT by nohav.com                                          |
| -author     | nohav.com                                                  |
| -contact    | nohav.com@gmail.com                                        |
| -homep...   | www.nohav.com                                              |
| -descrip... | Apps helps to use library cv2 on more user friendly level. |
| -max_re...  | ==>dict                                                    |
| -config     | ==>dict                                                    |
| -create     | ==>list                                                    |
| -update     | ==>dict                                                    |
| -modules    | ==>dict                                                    |
| -app        | ==>dict                                                    |

On the right side of the table mentioned above, there are three buttons for operations you can apply to the config file. The available operations are:

- add("+")
- edit
- remove("-")

Below the config table, there is a **"Config Detail"** area with three boxes

- Name: the key used in the config file (JSON).

Config detail

1

Name:

- Value: the value from the config file (JSON).

|        |    |
|--------|----|
| Value: | 13 |
|--------|----|

- Type: the type of the value – how it is stored in the config file (JSON).

|       |                                  |
|-------|----------------------------------|
| Type: | list<br>dict<br>string<br>number |
|-------|----------------------------------|

For more information about the available types, please refer to the "*Limitations*" chapter.

## Config tree

This tree shows each key-value pair from the config file in the same format (here named "Name" + "Value").

| Config      |                   |
|-------------|-------------------|
| Name        | Value             |
| .           | .                 |
| -name       | cvyt              |
| -version    | 0.0.1             |
| -title      | CVYT by nohav.co  |
| -author     | nohav.com         |
| -contact    | nohav.com@gmail   |
| -homep...   | www.nohav.com     |
| -descrip... | Apps helps to use |
| -max_re...  | ==>dict           |

This means you can encounter options such as:

- “name” == “cvyt” ==> key = “name”, value=“cvyt”(line from dict)
- “1” == “1” ==> value = 1 or “1”, value from list
- “empty\_dict” == “==>dict” ==> key = “empty\_dict”, value = dict object
- “empty\_list” == “==>list” ==> key = “empty\_list”, value = list object
- “..1” == “==>list” ==> key = index of item in that object, value = list object

That is just the tip of the iceberg. There are limitations on what can be added to what, the types you can use for values, and so on.

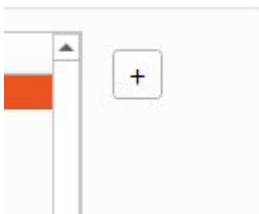
## Supported types

- Int, float, string
- List, dictionary

I will try to cover this topic at the end of this chapter (with examples).

## Add

Add a new key/value pair to the config file. Select where you want to apply this operation. For the "Add" operation, you need to place your cursor on the item "." or "..". The app will inform you if this operation is allowed on the selected item. The next step is to click the "+" button.



The next window should give you a hint about what to do and what actions are available.

Add to: dict

Name:

Keep in mind what you are adding when a value needs to be filled.

Value:

Keep in mind what you are adding when a value needs to be filled.

Type:

A value of this type will be stored as a string.

The "Add to:" line indicates where you are trying to add an item, which implies certain limitations. You can now fill in the "Name", "Value", and "Type" fields. Once you are done, click the "Ok" button.

## Examples

1. You want to add a key-value pair to the "dict"
  1. Fill "Name", "Value" and "Type" of value
2. You want to add a list or a dict to "dict"
  1. Fill "Name" and "Type" (it will take only these items)
3. You want to add an item to "list"

1. Fill “*Value*” and “*Type*”(only required items)
4. You want to add a “*list*” or a “*dict*” to a “*list*”
  1. Fill “*Type*”(it will add an empty list or dict)

**Note: Help messages are shown under each box. In case of a problem, you should see a help message guiding you the right way.**

## Edit

You can change the value to a different value or rename the key using the "Edit" button. Before clicking the button, select the item you want to modify.

The screenshot shows a configuration editor interface. At the top, there is a list of items with their keys and values: '-str4 ==>list', '-empty\_... ==>dict', '-empty\_... ==>list', '-2 ==>list', '-number ==>dict', and '-1 13'. The item '-1 13' is highlighted with an orange background. Below this list is a section titled 'Config detail' which contains a text input field with the value '1'.

Only simple types can be edited (string, number).

The screenshot shows a dialog box titled 'Edit from: str'. It contains three main sections: 'Name:' with a text input field containing '1'; 'Value:' with a text input field containing '13'; and 'Type:' with a dropdown menu showing 'list', 'dict', 'string' (selected and highlighted in orange), and 'number'. Below the 'Type:' dropdown, there is a message: 'A value of this type will be stored as a string.'

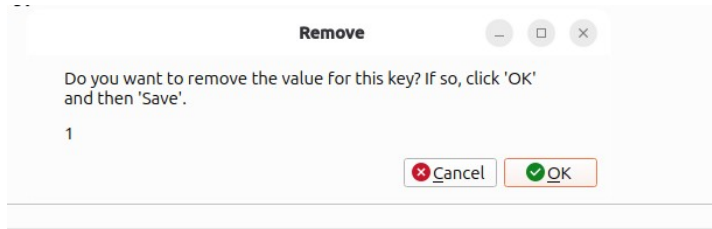
The rules and limitations are very similar to those for "Add". This means you should pay close attention to the help messages.

## Remove

The last option is "Remove". Select the item you want to remove.

This screenshot is identical to the one in the 'Edit' section, showing the configuration editor with the item '-1 13' selected and the 'Config detail' section below it.

The next step is to click the "-" button. This will open the "Are you sure?" window.



## Config tree, value combinations

Here, I try to cover most of the possibilities and options you may encounter while altering the config file.

The same logic applies to the "Add", "Edit", and "Remove" operations. As mentioned, pay close attention to the help messages under each box.

### Add

- You can add any type of item to a list:
  - List  $\Rightarrow$  the item will be added at the end of the list
    - You only need to specify "Type" = list
  - Dict  $\Rightarrow$  the item will be added at the end of the list
    - You only need to specify "Type" = dict
  - String  $\Rightarrow$  the item will be added at the end of the list
    - You must specify "Value" and "Type" = string
  - Number (int, float)  $\Rightarrow$  the item will be added at the end of the list
    - You must specify "Value" and "Type" = number
- Adding items to a dict:
  - List
    - Specify "Name" (key) and "Type" = list
  - Dict
    - Specify "Name" (key) and "Type" = dict
  - String
    - Specify "Name", "Value", and "Type" = string
  - Number(int, float)
    - Specify "Name", "Value", and "Type" = number

## **Edit**

- Number (int, float) and String
  - Select the appropriate item
  - Change the necessary fields ("Name", "Value", "Type")
- Other types (Dict, List) cannot be edited

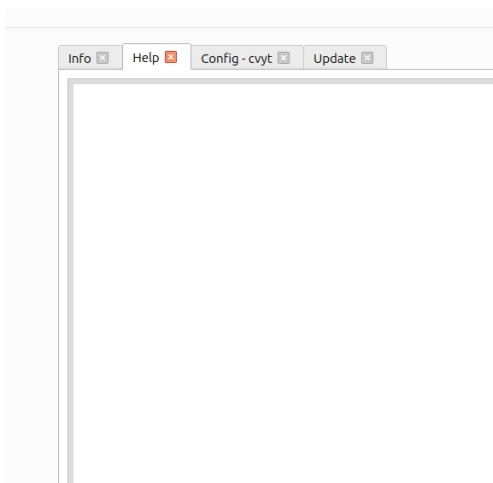
## **Remove**

- Removing items:
  - Simply select (click on) the item you want to remove, click "Remove", confirm the "Are you sure?" prompt, and the action is complete.
  - If you are removing a dict  $\Rightarrow$  all items within that dict will also be removed.
  - If you are removing an item from a dict  $\Rightarrow$  only that specific item (key + value) will be removed.
  - If you are removing an item from a list  $\Rightarrow$  only the item at that index will be removed (you should be able to remove an item at a specific index in case multiple identical values exist in the list).
  - If you are removing a list  $\Rightarrow$  all items within that list will be removed.

## **Help tab**

This tab, if you have chosen to use it, provides a more detailed help file.

For other operations, double-clicking "Help" in the main menu will open the help file in a new window of your default browser (note that this may take a few seconds).



## **Info tab**

This tab provides basic information about the app (homepage, contact, etc.).

Info 

Version: 0.0.1

Name: cvyt

Contact: nohav.com@gmail.com

Homepage: [www.nohav.com](http://www.nohav.com)

Description: Apps helps to use library cv2 on more user friendly level.

# Chapter 3

## Modules

Each "logic" is stored in a folder named after its unique module name, e.g., "split\_to\_frames". Every module has a defined structure, which is the same for all modules.

### Module content structure

```
- split_to_frames
  |- split_to_frames_widget.py
  |- split_to_frames_logic.py
  |- ...
  |- module_config.json
  |- help.pdf
```

### Module import/start

If the model is valid and listed among the available modules, you can use it.

We have already discussed how to import a module and add it to the tabs list, but here is what the app actually does:

The module is loaded and imported. As part of the import process, the app passes itself as ``main_widget`` and the ``app_config`` object as keyword arguments (kwargs).

From ``main_widget``, you can use the following methods:

- `open_help_window_browser` – expects arguments:
  - **help\_path** ⇒ path to the PDF help file to open using the default browser
- `open_config_widget` – expects arguments:
  - **config\_path** ⇒ path to the configuration file
  - **name** ⇒ name of the tab; should be unique (otherwise it may conflict with an existing tab), e.g., "Config - filters"

### Update module

To update modules, first place all module folders into a single folder and create an archive from it.

Open the app, go to the "Update" tab, and select the archive. Click the "Show" button to validate its contents.

If the content is valid and at least some subfolders (modules) contain the required structure, they will be displayed in the modules list. From this list, you can select which modules you want to update using the checkboxes.

Click "Update", verify that the update completed successfully, and then restart the app. After restarting, the new or updated modules should appear.



## Chapter 4

### Troubleshooting

Follow the chain of messages from the UI, combined with messages from the log file.

### *Logging*

When you start the app, a file named "*app\_cvyt.log*" is created. The log file uses a rotating log configuration with a maximum size of 1 MB and keeps up to three backup files.

### *Everything si broken*

Good luck :)

# Chapter 5

## Development

Add dependencies to "pyproject.toml". As a package manager, you can use Poetry.

## Tests

A basic test suite is included. Tests are implemented using "pytest".

## Changes, new ideas, cooperation

Anyone reading this documentation and using the application is welcome to join and help develop or extend it so we can get the most out of it. If you have an idea, fix, suggestion, or anything else, please don't hesitate to contact me. I'm always looking for cooperation and new ideas from a fresh point of view.

If you have already turned your idea into code, please create a pull request. I will try to review it as soon as possible and provide feedback.

In the section below, you can find some points that need to be **DONE** and others that **CAN** be improved or implemented.

## Future

### *Technical debt & New ideas*

1. Polish the UI (issues with the flexibility of items)
2. Add a log file overview
3. Add more tests
4. Define a way to add dependencies related to modules
  1. Approach used by "Starter" (venv)
5. Create and add new modules